

Tool Calling with AI Agents

Chapter 2: Agentic AI Deep Dive — Model · Tools · Memory

What You Will Build Today

- Practical 1 — Single Tool Agent: A weather agent using one tool
 - Practical 2 — Multi-Tool Agent: Auto-selects from 3 tools (weather, calculator, time)
 - Practical 3 — Memory + Tools: A stateful chatbot that remembers context
-

Step 0 — Install & Setup

Install the Groq SDK and configure your API client before running any practicals.

Install Groq

```
!pip install groq -q
```

Configure API Client

```
GROQ_API_KEY = "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"

from groq import Groq

client = Groq(api_key=GROQ_API_KEY)
MODEL = "llama-3.1-8b-instant"
```

⚠ Security Note: Never hard-code API keys in production code. Use environment variables or a secrets manager instead.

Quick Recap — Why Do We Need Tools?

Run the cell below to see the model fail on real-world questions that require live data.

```
questions = [
    "What is the weather in Chennai right now?",
    "What is today's date?",
    "What is the current price of gold in India?"
]

print(" Model WITHOUT Tools\n" + "="*50)
for q in questions:
    response = client.chat.completions.create(
        model=MODEL,
        messages=[{"role": "user", "content": q}],
        max_tokens=80
    )
    answer = response.choices[0].message.content.strip()
    print(f"\n ? {q}")
    print(f"🤖 {answer[:200]}")
    print("-"*50)
```

💡 Key Insight: The model can't answer these because it has no real-time data access. Tools bridge this gap.

Practical 1 — Single Tool: Weather Agent

Build a weather agent that follows this pipeline:

Flow: User Input → Model Detects Need → Tool Invoked → Result Returned → Model Responds

1a. Define the Weather Tool Function

This function simulates a Weather API. In production, you would call OpenWeatherMap or a similar service.

```
import json

def get_weather(city: str) -> dict:
    """Simulates a Weather API call."""
```

```

weather_data = {
    "chennai": {"temp": 34, "condition": "Sunny", "humidity": 78,
"unit": "°C"},
    "mumbai": {"temp": 29, "condition": "Cloudy", "humidity": 85,
"unit": "°C"},
    "delhi": {"temp": 38, "condition": "Hazy", "humidity": 45,
"unit": "°C"},
    "bangalore": {"temp": 24, "condition": "Rainy", "humidity": 90,
"unit": "°C"},
    "coimbatore": {"temp": 31, "condition": "Partly Cloudy", "humidity": 70,
"unit": "°C"},
    "kolkata": {"temp": 32, "condition": "Humid", "humidity": 88,
"unit": "°C"},
}
key = city.lower().strip()
if key in weather_data:
    data = weather_data[key]
    return {
        "city": city.title(),
        "temperature": f"{data['temp']}{data['unit']}",
        "condition": data["condition"],
        "humidity": f"{data['humidity']}%",
        "status": "success"
    }
return {"status": "error", "message": f"City '{city}' not found"}

```

Test the Function Directly

```

print(get_weather("Chennai"))
print(get_weather("Mumbai"))

```

```

{'city': 'Chennai', 'temperature': '34°C', 'condition': 'Sunny', 'humidity':
'78%', 'status': 'success'}
{'city': 'Mumbai', 'temperature': '29°C', 'condition': 'Cloudy', 'humidity':
'85%', 'status': 'success'}

```

1b. Define the Tool Schema

The schema tells the model what tools are available and how to call them. It follows the OpenAI function-calling format.

```

weather_tool_schema = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get the current weather for any city in India.",
            "parameters": {
                "type": "object",
                "properties": {
                    "city": {
                        "type": "string",

```

```

        "description": "City name e.g. Chennai, Mumbai, Delhi"
    },
    },
    "required": ["city"]
}
}
]

```

1c. Build the Weather Agent

The agent follows a 5-step loop: receive input → decide → call tool → get result → respond.

```

def weather_agent(user_question: str, verbose: bool = True):
    """Weather agent that uses tool calling to answer weather questions."""

    if verbose:
        print(f"\n{'='*55}")
        print(f"STEP 1 – User Input: {user_question}")
        print(f"{'='*55}")

    messages = [{"role": "user", "content": user_question}]

    # First call: let the model decide if a tool is needed
    response = client.chat.completions.create(
        model=MODEL,
        messages=messages,
        tools=weather_tool_schema,
        tool_choice="auto"
    )

    msg = response.choices[0].message
    finish_reason = response.choices[0].finish_reason

    if verbose:
        print(f"\nSTEP 2 – Model Decision: finish_reason = '{finish_reason}'")

    if finish_reason == "tool_calls" and msg.tool_calls:
        tool_call = msg.tool_calls[0]
        tool_name = tool_call.function.name
        tool_args = json.loads(tool_call.function.arguments)

        if verbose:
            print(f"STEP 3 – Tool Invoked : {tool_name}")
            print(f"Arguments : {tool_args}")

        tool_result = get_weather(**tool_args)

        if verbose:
            print(f"STEP 4 – Tool Result : {tool_result}")

        # Add tool result to conversation and get final answer
        messages.append(msg)
        messages.append({

```

```

        "role": "tool",
        "tool_call_id": tool_call.id,
        "content": json.dumps(tool_result)
    })

    final = client.chat.completions.create(model=MODEL, messages=messages)
    final_answer = final.choices[0].message.content
else:
    final_answer = msg.content

if verbose:
    print(f"\nSTEP 5 - Final Answer: {final_answer}")

return final_answer

```

Run the Agent

```

weather_agent("What is the weather like in Chennai today?")
weather_agent("Is it raining in Bangalore? Should I carry an umbrella?")

```

Practical 2 — Multi-Tool Agent

Now we give the model three tools and watch it automatically pick the right one for each question.

Pipeline: Question → Model reads all tool descriptions → Picks the best tool → Calls it

2a. Tool 2 — Calculator

```

def calculate(expression: str) -> dict:
    """Safely evaluates a math expression."""
    try:
        allowed = set("0123456789+-*/(). ")
        if not all(c in allowed for c in expression):
            return {"error": "Invalid characters in expression"}
        result = eval(expression)
        return {"expression": expression, "result": result, "status": "success"}
    except Exception as e:
        return {"error": str(e), "status": "failed"}

```

2b. Tool 3 — Time Zone Converter

```
def convert_time(time_str: str, from_timezone: str, to_timezone: str) -> dict:
    """Converts time between timezones (IST, UTC, EST, PST, CET)."""
    offsets = {
        "IST": 5.5, "UTC": 0, "EST": -5, "PST": -8, "CET": 1
    }
    try:
        hour, minute = map(int, time_str.replace(":", " ").split()[:2])
        from_off = offsets.get(from_timezone.upper(), 0)
        to_off = offsets.get(to_timezone.upper(), 0)
        diff = to_off - from_off
        new_hour = int((hour + diff) % 24)
        return {
            "original": f"{time_str} {from_timezone}",
            "converted": f"{new_hour:02d}:{minute:02d} {to_timezone}",
            "difference_hours": diff,
            "status": "success"
        }
    except:
        return {"status": "error", "message": "Could not parse time"}
```

Test the Tools

```
print("Calculator:  ", calculate("(45 * 12) + 300"))
print("Time convert: ", convert_time("09:00", "IST", "EST"))
```

```
Calculator:    {'expression': '(45 * 12) + 300', 'result': 840, 'status':
'success'}
Time convert: {'original': '09:00 IST', 'converted': '22:00 EST',
'difference_hours': -10.5, 'status': 'success'}
```

2c. Register All Tools

```
ALL_TOOLS = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get current weather for any city in India.",
            "parameters": {
                "type": "object",
                "properties": {"city": {"type": "string"}},
                "required": ["city"]
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "calculate",
            "description": "Perform math calculations. Use for arithmetic,
percentages, or formulas.",
            "parameters": {
                "type": "object",
```

```

        "properties": {"expression": {"type": "string"}},
        "required": ["expression"]
    }
}
},
{
    "type": "function",
    "function": {
        "name": "convert_time",
        "description": "Convert time between timezones. IST, UTC, EST, PST,
CET supported.",
        "parameters": {
            "type": "object",
            "properties": {
                "time_str": {"type": "string"},
                "from_timezone": {"type": "string"},
                "to_timezone": {"type": "string"}
            },
            "required": ["time_str", "from_timezone", "to_timezone"]
        }
    }
}
]

# Map tool names to Python functions
TOOL_MAP = {
    "get_weather": get_weather,
    "calculate": calculate,
    "convert_time": convert_time
}

```

2d. Build the Multi-Tool Agent

```

def multi_tool_agent(user_question: str):
    """Agent that picks the right tool from 3 options automatically."""
    print(f"\n{'='*55}")
    print(f" User: {user_question}")
    print(f"{'='*55}")

    messages = [{"role": "user", "content": user_question}]
    response = client.chat.completions.create(
        model=MODEL,
        messages=messages,
        tools=ALL_TOOLS,
        tool_choice="auto"
    )

    msg = response.choices[0].message
    finish_reason = response.choices[0].finish_reason

    if finish_reason == "tool_calls" and msg.tool_calls:
        tool_call = msg.tool_calls[0]
        tool_name = tool_call.function.name
        tool_args = json.loads(tool_call.function.arguments)
        print(f" Model chose tool : {tool_name}")
        print(f" Arguments      : {tool_args}")

```

```

    tool_fn      = TOOL_MAP[tool_name]
    tool_result = tool_fn(**tool_args)
    print(f" Tool result      : {tool_result}")

    messages.append(msg)
    messages.append({
        "role":      "tool",
        "tool_call_id": tool_call.id,
        "content":    json.dumps(tool_result)
    })
    final = client.chat.completions.create(model=MODEL, messages=messages)
    answer = final.choices[0].message.content
else:
    print(" Model answered directly (no tool needed)")
    answer = msg.content

print(f"\n Agent: {answer}")
return answer

```

Test All Three Tools

```

multi_tool_agent("What's the weather in Delhi today?")
multi_tool_agent("If a course costs ₹4500 and there's a 15% discount, what is the
final price?")
multi_tool_agent("If my meeting is at 10:00 AM IST, what time is that in the USA
(EST)?")
multi_tool_agent("What is the capital of Tamil Nadu?") # No tool needed

```

User: What's the weather in Delhi today?

Model chose tool: get_weather | Arguments: {'city': 'Delhi'}
Tool result: {'city': 'Delhi', 'temperature': '38°C', 'condition': 'Hazy',
'humidity': '45%'}
Agent: The current weather in Delhi is 38°C with a hazy condition and 45%
humidity.

User: If a course costs ₹4500 with 15% discount, what is the final price?

Model chose tool: calculate | Arguments: {'expression': '4500 * (1 - 15/100)'}
Tool result: {'result': 3825.0, 'status': 'success'}
Agent: The final price after a 15% discount is ₹3825.

User: If my meeting is at 10:00 AM IST, what time is that in EST?

Model chose tool: convert_time | Arguments:
{'time_str': '10:00', 'from': 'IST', 'to': 'EST'}
Tool result: {'converted': '23:00 EST', 'difference_hours': -10.5}
Agent: 10:00 AM IST = 23:00 (11:00 PM) EST (IST is 10.5 hours ahead of EST).

Practical 3 — Memory + Tools: Stateful Chatbot

A stateful chatbot that remembers your city across the conversation and answers weather questions without asking every time.

This combines two concepts: Short-term memory (conversation history) + Tool calling (get_weather).

```
class WeatherChatbot:
    """A stateful chatbot that remembers the conversation (short-term memory)."""

    def __init__(self):
        self.history = []
        self.system = (
            "You are a helpful weather assistant for India. "
            "When asked about weather, always use the get_weather tool. "
            "Remember what city the user mentioned earlier in the conversation."
        )

    def chat(self, user_input: str):
        print(f"\n👤 You : {user_input}")
        self.history.append({"role": "user", "content": user_input})

        messages = [{"role": "system", "content": self.system}] + self.history
        response = client.chat.completions.create(
            model=MODEL, messages=messages,
            tools=weather_tool_schema, tool_choice="auto"
        )

        msg = response.choices[0].message
        finish_reason = response.choices[0].finish_reason

        if finish_reason == "tool_calls" and msg.tool_calls:
            tool_call = msg.tool_calls[0]
            tool_args = json.loads(tool_call.function.arguments)
            tool_result = get_weather(**tool_args)
            print(f" [Tool called: get_weather({tool_args})]")

            self.history.append(msg)
            self.history.append({
                "role": "tool",
                "tool_call_id": tool_call.id,
                "content": json.dumps(tool_result)
            })

            messages = [{"role": "system", "content": self.system}] + self.history
            final = client.chat.completions.create(model=MODEL,
            messages=messages)
            reply = final.choices[0].message.content
        else:
            reply = msg.content

        self.history.append({"role": "assistant", "content": reply})
        print(f"🤖 Agent : {reply}")
```

```
print(f"    [Memory: {len(self.history)} messages stored]")
return reply
```

Run the Stateful Chatbot

```
bot = WeatherChatbot()

bot.chat("Hi! I live in Coimbatore.")
bot.chat("What's the weather like today?")
bot.chat("Should I take an umbrella when I go out?")
```

```
👤 You    : Hi! I live in Coimbatore.
🤖 Agent  : Got it! Nice to meet you. I'll remember Coimbatore for your weather
queries.
           [Memory: 2 messages stored]

👤 You    : What's the weather like today?
           [Tool called: get_weather({'city': 'Coimbatore'})]
🤖 Agent  : The current weather in Coimbatore is Partly Cloudy, 31°C, humidity 70%.
           [Memory: 6 messages stored]

👤 You    : Should I take an umbrella when I go out?
🤖 Agent  : With Partly Cloudy skies and 70% humidity in Coimbatore, there's a
chance of rain.
           It's a good idea to carry an umbrella just in case!
           [Memory: 8 messages stored]
```

Summary — What You Built Today

Practical	Component
Model fails on real questions	Model Limitations
Single tool: Weather Agent	Tool Workflow
Multi-tool: auto tool selection	Tool Examples (Calculator, Time)
Memory + Tool chatbot	Short-term Memory

The Intelligence Stack

Model alone → LIMITED (no real-world data access)

Model + Tools → Can ACT in the world

Model + Tools + Memory → INTELLIGENT AGENT

Key Concepts Covered

- Tool schema definition (OpenAI function-calling format)
- `finish_reason == 'tool_calls'` — how the model signals a tool request
- Two-turn conversation pattern: user → model → tool → model → user
- Multi-tool routing: the model picks the right tool from descriptions alone
- Short-term memory via message history accumulation